

12 · Pruebas y calidad

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 12-testing-and-quality.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Resumen
2. Cómo se ejecutan
3. Inventario por módulo
4. Qué protege cada grupo de tests
5. Fixtures y datos de prueba
6. Módulos y funciones sin cobertura
7. Análisis estático y estilo
8. Integración continua
9. Criterios de aceptación observables
10. Casos límite relevantes ya cubiertos
11. Casos límite ****no**** cubiertos
12. Pruebas faltantes, priorizadas
13. Mejoras de proceso recomendadas
14. Resultados de la ejecución de este análisis

Qué se prueba, cómo se ejecuta, qué queda sin cubrir y qué pruebas faltan, ordenadas por prioridad. Los 112 tests de este documento se ejecutaron en este análisis.

1. Resumen

Indicador	Valor
Tests unitarios	112
Tests de integración (tests/)	0 — no existe el directorio
Tests de interfaz	0
Módulos con tests	20 de 23 archivos .rs
Módulos sin tests	main.rs, meta.rs, services/mod.rs
Resultado de la última ejecución	112 passed; 0 failed; 0 ignored, 1,47 s
Análisis estático	cargo clippy -- all-targets -- all-features -- -D warnings, sin advertencias
Formato	cargo fmt --all -- --check, sin diferencias
Lint de documentación	markdownlint- cli2 en la CI
Cobertura medida	No se mide: no hay tarpaulin, llvm-cov ni informe de cobertura

2. Cómo se ejecutan

```
cargo test --all-features          # los 112
cargo test --all-features -- --nocapture # con stdout, igual que la CI
cargo test services::rules         # solo un módulo
cargo test la_huella_agrupa        # un test por nombre parcial
./scripts/ci-local.sh              # formato + clippy + tests + dos builds + humo
```

Los tests están **junto al código**, en módulos `#[cfg(test)]` al final de cada archivo. Es la convención idiomática de Rust y tiene una ventaja concreta aquí: permiten probar funciones privadas (`severity_for`, `age_severity`, `first_line`, `expand_home`, `ip_sort_key`) sin hacerlas públicas solo para el test.

3. Inventario por módulo

Módulo	Tests	Qué fijan
services/anomaly .rs	14	Las ocho heurísticas y sus guardas contra falsos positivos
services/rules. rs	11	Clasificación de procesos, orden de alertas, derivación de incidentes
cli.rs	8	Parseo de banderas, códigos de salida, recorte multibyte

services/netscan.rs	8	Parseo ARP, normalización de MAC, caso crítico de la puerta de enlace
services/macros.rs	7	Clasificación de codesign y declaración del entorno
services/network.rs	6	Parseo de Lsof , IP públicas y privadas, agrupación por PID
services/tcc.rs	6	Ambos esquemas de TCC, severidad y filtro de baseline
services/ai.rs	6	Guardas de la IA, parseo de respuesta, contenido del payload
app.rs	5	Buffer de tendencia, porcentajes, secciones únicas
models.rs	5	Orden de severidades y pesos de riesgo
services/launchd.rs	5	Clasificación de entradas de persistencia
services/security.rs	5	Regla «desconocido no es verde», antigüedad de firmas
services/temp_scan.rs	5	Umbral de tamaño y simulacro de limpieza
services/baseline.rs	4	Eventos por tipo de cambio y su evidencia
services/report.rs	4	Secciones del reporte, escapado y declaración de privacidad
config.rs	3	Coherencia de los valores por defecto y viaje a JSON
services/inspector.rs	3	Conversiones y efecto de la salud del agente
services/resilience.rs	3	Huella de configuración y serialización del estado
i18n.rs	2	Traducción según idioma activo
services/persistence.rs	2	Estabilidad de la clave de baseline

4. Qué protege cada grupo de tests

4.1 Los que fijan decisiones de producto

Estos no comprueban código: **comprueban criterios**. Si alguien los rompe, ha cambiado el producto, no solo la implementación.

Test	Criterio que fija
una_cpu_alta_por_si_sola_no_es_critica	Compilar no debe pintarse de rojo

un_pico_de_cpu_ no_dispara_nada	Sostenido, no instantáneo
control_ desconocido_no_ se_pinta_de_ verde	«No lo sé» nunca es verde
permiso_denegado_ _nunca_es_ advertencia	Un permiso denegado no es un hallazgo
salida_ desconocida_no_ asume_confianza	Lo que no se entiende no se da por bueno
el_payload_solo_ lleva_el_ incidente_ resumido	La IA no recibe la captura completa
la_ia_ desactivada_ falla_sin_tocar_ la_red	Apagado significa apagado
el_reporte_ declara_que_nada_ _sale_del_equipo	La promesa de privacidad está en la salida
el_simulacro_ nunca_marca_ borrado_real	dry_run no borra
un_navegador_ instalado_con_ normalidad_no_ dispara_trafico_ inusual	Sin falsos positivos con software normal
un_binario_sin_ firmar_en_ applications_si_ dispara_trafico_ inusual	...pero la señal se mantiene donde importa
gateway_nuevo_es_ _incidente_ critico	Cambio de MAC del router = crítico
persistencia_ nueva_y_ sospechosa_ escala_a_critica	Dos señales independientes escalan
la_clave_ignora_ el_comando	Cambiar el comando es modificación, no reemplazo
la_huella_agrupa_ _incidentes_ equivalentes	No se persiste el mismo hallazgo cada segundo

4.2 Los que protegen de errores concretos

Test	Error que evita
recorta_ respetando_ caracteres_ multibyte (x2)	Pánico al cortar cadenas con acentos
una_bandera_sin_ valor_no_entra_ en_panico	Pánico con una bandera al final de la línea
normaliza_ octetos_sin_cero_ a_la_izquierda	Falsos cambios de MAC en la baseline
parsea_nombres_ de_proceso_con_ espacios	Parseo roto con Google Chrome
ordena_ips_ numericamente	.10 antes que .2
json_vacio_cae_a_ defaults_sin_ romper	Configuración parcial que rompe el arranque
un_estado_vacio_ no_rompe_la_ deserializacion	Estado de agente vacío
medir_directorio_ inexistente_ devuelve_cero	Error al medir una ruta que no existe
epoch_cero_no_ produce_fecha	Fecha inventada a partir de un cero
las_barras_no_ rompen_las_ tablas	Tabla Markdown rota por un \
varias_ instancias_del_ mismo_nombre_no_ son_una_ reaparicion	Falso positivo con procesos multi-instancia
el_buffer_de_ tendencia_no_ crece_sin_limite	Fuga de memoria en las series
el_porcentaje_ tolera_total_ cero	División por cero

4.3 El único test que toca el sistema real

`services::macos::tests::el_entorno_real_encuentra_las_utilidades_base_de_macos` comprueba que `launchctl` existe en el equipo donde corre. Es el único test no determinista del conjunto y **fallaría fuera de macOS**, lo que es coherente: el producto solo compila para macOS en la práctica.

Todos los demás son puros: usan muestras literales (una salida de `lsof`, una tabla ARP, una respuesta de `codesign`) y estructuras construidas a mano.

5. Fixtures y datos de prueba

No hay directorio de fixtures ni archivos de datos. Las muestras viven **como constantes en el propio test**:

Constante	Módulo	Qué reproduce
<code>SAMPLE</code>	<code>network.rs</code>	Salida de <code>lsof -F</code> con tres sockets, uno con nombre de proceso con espacios
<code>ARP_SAMPLE</code>	<code>netscan.rs</code>	Cuatro líneas de <code>arp -a -n</code> , dos válidas y dos de ruido
Cadenas de <code>codesign</code>	<code>macos.rs</code>	Cinco variantes de salida real
<code>base_entry()</code>	<code>launchd.rs</code>	Entrada de persistencia típica de una aplicación
<code>snapshot_with_anomaly()</code>	<code>rules.rs</code>	Captura con una anomalía de severidad dada
<code>snapshot()</code>	<code>report.rs</code>	Captura con alerta que contiene una barra vertical

Es una decisión coherente con el tamaño del proyecto: menos archivos que mantener, y la muestra se lee junto al test que la usa.

6. Módulos y funciones sin cobertura

6.1 Sin ningún test

Elemento	Motivo	Riesgo
<code>main.rs</code>	Punto de entrada; probarlo exige lanzar el proceso	Bajo: 20 líneas de despacho
<code>meta.rs</code>	Solo constantes	Nulo
<code>services/mod.rs</code>	Solo declaraciones	Nulo

6.2 Cubiertas parcialmente

Función	Qué se prueba	Qué no
<code>InspectorService::collect_snapshot</code>	Nada directamente	Toda la secuencia de captura: es el hueco más grande
<code>PersistenceStore</code>	Solo <code>persistence_entry_key</code>	Ninguna operación SQL real
<code>baseline::diff_surface</code>	Solo <code>surface_change_event</code>	El diff contra una base real
<code>launchd::scan_persistence</code>	Solo <code>classify_entry</code>	Lectura y parseo de plists reales
<code>security::scan_controls</code>	Solo helpers	La consulta a los seis controles
<code>tcc::scan</code>	Solo helpers	La lectura de una base TCC

<code>temp_scan::clean_user_caches</code>	Solo el simulacro	El borrado real
<code>report::save_report</code>	Solo <code>build_report</code>	La escritura en disco
<code>ai::post_json</code>	Nada	La petición HTTP (razonable: es red externa)
Todo <code>app.rs</code> salvo helpers	Cinco helpers	El dibujado y el hilo de trabajo

El patrón es claro y consistente: está probado todo lo que es función pura, y sin probar todo lo que toca el sistema de archivos, la base de datos o la red. Es una elección defendible —esas partes necesitarían un entorno controlado— pero deja fuera precisamente la orquestación, que es donde se juntan las piezas.

7. Análisis estático y estilo

Herramienta	Configuración	Estado
<code>rustfmt</code>	<code>rustfmt.toml</code> : edición 2021, ancho 100, saltos Unix	En verde
<code>clippy</code>	<code>--all-targets --all-features --D warnings</code>	En verde, sin excepciones
<code>markdownlint-cli2</code>	<code>.markdownlint-cli2.jsonc</code>	En verde en CI
<code>cargo audit</code>	No configurado	—
Cobertura	No configurada	—

Cero `#[allow(...)]` de conveniencia, con dos excepciones justificadas en el código:

- `#![cfg_attr(not(feature = "gui"), allow(dead_code))]` en `main.rs`, con comentario que explica que el código lo usa la otra edición.
- `#[allow(clippy::too_many_arguments)]` en `anomaly::process_event`, un constructor común de eventos con nueve parámetros.

Que `clippy` pase con `-D warnings` sin silenciar lints es un indicador de calidad significativo en un proyecto de 13 000 líneas.

8. Integración continua

`.github/workflows/ci.yml`, dos jobs:

Job	Runner	Pasos
<code>docs</code>	<code>ubuntu-latest</code>	<code>npx --yes markdownlint-cli2</code>
<code>validate</code>	<code>macos-latest</code>	Toolchain + caché → versiones → <code>fmt --check</code> → <code>clippy -D warnings</code> → <code>test --all-features -- --nocapture</code> → build CLI-only → build completa → humo de la CLI → artefacto

El humo de la CLI ejecuta cuatro comandos reales en el runner:


```
./target/release/rootcause --version
./target/release/rootcause --help > /dev/null
./target/release/rootcause security
./target/release/rootcause persistence
```

Es más valioso de lo que parece: comprueba que el binario arranca, que el motor se inicializa, que se crea la carpeta de datos, que el esquema SQLite se aplica y que dos superficies reales responden en un macOS limpio. Es, de hecho, **la única prueba de integración del proyecto**, aunque no esté escrita como test.

9. Criterios de aceptación observables

Deducidos de la CI y de los scripts, no declarados como tales en el repositorio:

1. El código está formateado según `rustfmt.toml`.
2. `clippy` no emite ni una advertencia con todas las features.
3. Los 112 tests pasan.
4. Las dos ediciones compilan en release.
5. El binario responde a `--version`, `--help`, `security` y `persistence`.
6. Todos los Markdown pasan `markdownlint-cli2`.

`scripts/ci-local.sh` reproduce los cinco primeros y termina con «Todo en verde. Listo para empujar».

10. Casos límite relevantes ya cubiertos

Caso	Test
Cadena con acentos en una tabla	<code>recorta_</code> <code>respetando_</code> <code>caracteres_</code> <code>multibyte</code>
Bandera sin valor	<code>una_bandera_sin_</code> <code>valor_no_entra_</code> <code>en_panico</code>
JSON de configuración vacío	<code>json_vacio_cae_a</code> <code>_defaults_sin_</code> <code>romper</code>
MAC con octetos cortos	<code>normaliza_</code> <code>octetos_sin_cero</code> <code>_a_la_izquierda</code>
Entradas ARP incompletas y de difusión	<code>parsea_vecinos_y</code> <code>_descarta_ruido</code>
Proceso con nombre compuesto	<code>parsea_nombres_</code> <code>de_proceso_con_</code> <code>espacios</code>
CGNAT y link-local	<code>ips_privadas_y_</code> <code>loopback_no_son_</code> <code>publicas</code>
IPv6 entre corchetes	<code>extrae_ip_de_</code> <code>extremos_v4_y_v6</code>
Directorio inexistente	<code>medir_directorio</code> <code>_inexistente_</code> <code>devuelve_cero</code>

Epoch cero en TCC	<code>epoch_cero_no_produce_fecha</code>
Esquema TCC heredado	<code>decodifica_el_esquema_hereditado</code>
Salida de <code>codesign</code> no reconocida	<code>salida_desconocida_no_asume_confianza</code>
Respuesta de IA sin <code>choices</code>	<code>una_respuesta_sin_choices_es_error_claro</code>
Captura sin hallazgos	<code>una_captura_sana_no_genera_incidente</code>

11. Casos límite no cubiertos

Caso	Qué podría pasar	Prioridad
Umbral invertido (<code>critical < warning</code>)	La rama de aviso queda inalcanzable	Media
<code>refresh_interval_secs = 0</code>	La GUI fuerza 2 s; el CLI no lo usa	Baja
Plist con <code>KeepAlive</code> como diccionario	Cubierto en el código, sin test	Media
Plist con <code>ProgramArguments</code> vacío	Comando vacío en la baseline	Media
Baseline con miles de entradas	Rendimiento del diff	Baja
SQLite bloqueado por otro proceso	Error de escritura → alerta	Media
Directorio de datos sin permiso de escritura	El servicio no arranca	Media
<code>TCC.db</code> con esquema desconocido	<code>PRAGMA</code> no encuentra columnas esperadas	Media
Reloj del sistema hacia atrás	Antigüedad negativa (ya se satura con <code>max(0)</code>)	Baja
Proceso que muere entre <code>ps</code> y <code>sysinfo</code>	Datos parciales de ese PID	Baja

12. Pruebas faltantes, priorizadas

Prioridad alta

- `PersistenceStore` sobre una base temporal.** Crear el store en un directorio temporal y probar: creación de esquema, inserción y recorte (`trim_table`), deduplicación por huella, y el ciclo completo `replace_baseline` → `load_baseline` → `diff_surface`. Es el componente con más lógica sin cobertura y el que sostiene toda la detección de cambios.
- `baseline::diff_surface` de punta a punta.** Con el store real: primera siembra silenciosa, detección de `Added/Modified/Removed`, y que los cambios sigan reportándose en la captura siguiente (la propiedad «pegajosa» no tiene ningún test).
- `launchd::parse_launch_plist` con plists sintéticos.** Escribir plists en un directorio temporal y comprobar: `Program` y `ProgramArguments` combinados, `KeepAlive` como diccionario, `StartInterval` inválido, plist corrupto y plist sin `Label`.

Prioridad media

- 1. `temp_scan::clean_user_caches` real, sobre un árbol temporal: que respete el corte de 24 horas, que no salga de la raíz y que cuente correctamente lo saltado.
- 2. `tcc::read_database` con bases sintéticas de ambos esquemas, creadas con `rusqlite`.
- 3. Validación de configuración: test que compruebe que umbrales invertidos se detectan (y, antes, decidir si deben corregirse o solo advertirse).
- 4. `report::save_report` en un directorio temporal.

Prioridad baja

- 1. Test de humo del ciclo completo `collect_snapshot` con un `InspectorService` apuntando a un directorio de datos temporal. Requiere parametrizar `meta::APP_DIR`, que hoy es una constante: es un cambio de diseño, no solo un test.
- 2. Pruebas de la interfaz con el arnés de pruebas de egui.
- 3. Comprobación de rendimiento del diff con baselines grandes.

Lo que no merece la pena probar

- El dibujado de widgets: coste alto, valor bajo, cambia con cada ajuste visual.
- La petición HTTP real a la IA: depende de un tercero; lo que importa —las guardas y el contenido del payload— ya está cubierto.

13. Mejoras de proceso recomendadas

Mejora	Beneficio	Coste
Añadir <code>cargo audit</code> a la CI	Vigila vulnerabilidades de las 326 dependencias	Bajo
Medir cobertura con <code>cargo llvm-cov</code>	Da un número objetivo y detecta huecos nuevos	Bajo
Fijar la versión de <code>markdownlint-cli2</code>	Evita que una versión nueva rompa la CI sin cambios en el repositorio	Muy bajo
Añadir un job que compile en una segunda versión de Rust	Detecta lints nuevos antes de que rompan la CI	Bajo
Directorio <code>tests/</code> con las pruebas de alta prioridad	Cubre el hueco de persistencia y baseline	Medio

La cuarta merece un comentario: el historial del repositorio incluye un commit titulado «Corregir dos lints que solo aparecen en *rustc 1.97 (el del runner)*», es decir, **ya ha ocurrido** que la CI detectara antes que el equipo local un lint nuevo.

14. Resultados de la ejecución de este análisis

Equipo: macOS 26.3.1 (25D2128), Apple Silicon.

Comando	Resultado
<code>cargo fmt --all</code> <code>-- --check</code>	Código de salida 0, sin diferencias

<code>cargo clippy --all-targets --all-features -- -D warnings</code>	Código de salida 0, sin advertencias
<code>cargo test --all -features</code>	112 passed; 0 failed; 0 ignored; 0 measured, 1,47 s
<code>cargo build --no-default-features</code>	Código de salida 0
<code>./target/debug/rootcause security</code>	Seis controles con evidencia real
<code>./target/debug/rootcause tcc</code>	Declara la falta de Acceso total al disco, como está diseñado
<code>./target/debug/rootcause config show --json</code>	Configuración por defecto idéntica a la documentada

Estos cuatro últimos comandos son, en la práctica, la misma prueba de humo que ejecuta la CI.

Siguiente lectura recomendada: [13 · Despliegue y operación.](#)